Summarize

Chapter 9. Data Ownership and Distributed Transactions

Friday, December 10 09:12



While the database team worked on decomposing the monolithic Sysops Squad database, the Sysops Squad development team, along with Addison, the Sysops Squad architect, started to work on forming bounded contexts between the services and the data, assigning table ownership to services in the process.

"Why did you add the expert profile table to the bounded context of the Ticket Assignment service?" asked Addison.

"Because," said Sydney, "the ticket assignment relies on that table for the assignment algorithms. It constantly queries that table to get the expert's location and skills information."

"But it only does queries to the expert table," said Addison. "The User Maintenance service contains the functionality to perform database updates to maintain that information. Therefore, it seems to me the expert profile table should be owned by the User Maintenance service and put within that bounded context."

"I disagree," said Sydney. "We simply cannot afford for the assignment service to make remote calls to the User Maintenance service for every query it needs. It simply won't work."

"In that case, how to you see updates occurring to the table when an expert acquires a new skill or changes their service location? And what about when we hire a new expert?" asked Addison. "How would that work?"

"Simple," said Sydney. "The User Maintenance service can still access the expert table. All it would need to do is connect to a different database. What's the big deal about that?"

"Don't you remember what Dana said earlier? It's OK for multiple services to connect to the same database schema, but it's not OK for a service to connect to multiple databases or schemas. Dana said that was a no-go and would not allow that to happen," said Addison.

"Oh, right, I forgot about that rule. So what do we do?" asked Sydney. "We have one service that needs to do occasional updates, and an entirely different service in an entirely different domain to do frequent reads from the table."

"I don't know what the right answer is," said Addison. "Clearly this is going to require more collaboration between the database team and us to figure these things out. Let me see if Dana can provide any advice on this."

Once data is pulled apart, it must be stitched back together to make the system work. This means figuring out which services own what data, how to manage distributed transactions, and how services can access data they need (but no longer own). In this chapter, we explore the ownership and transactional aspects of putting distributed data back together.

Assigning Data Ownership

After breaking apart data within a distributed architecture, an architect must determine which services own what data. Unfortunately, assigning data ownership to a service is not as easy as it sounds, and becomes yet another hard part of software architecture.

The general rule of thumb for assigning table ownership states that services that perform write operations to a table own that table. While this general rule of thumb works well for single ownership (only one service ever writes to a table), it gets messy when teams have joint ownership (multiple services do writes to the same table) or even worse, common ownership (most or all services write to the table).

TIP

The general rule of thumb for data ownership is that the service that performs write operations to a table is the owner of that table. However, joint ownership makes this simple rule complex!

To illustrate some of the complexities with data ownership, consider the example illustrated in Figure 9-1 showing three services: a Wishlist Service that manages all of the customer wish lists, a Catalog Service that maintains the product catalog, and an Inventory Service that maintains the inventory and restocking functionality for all products in the product catalog.

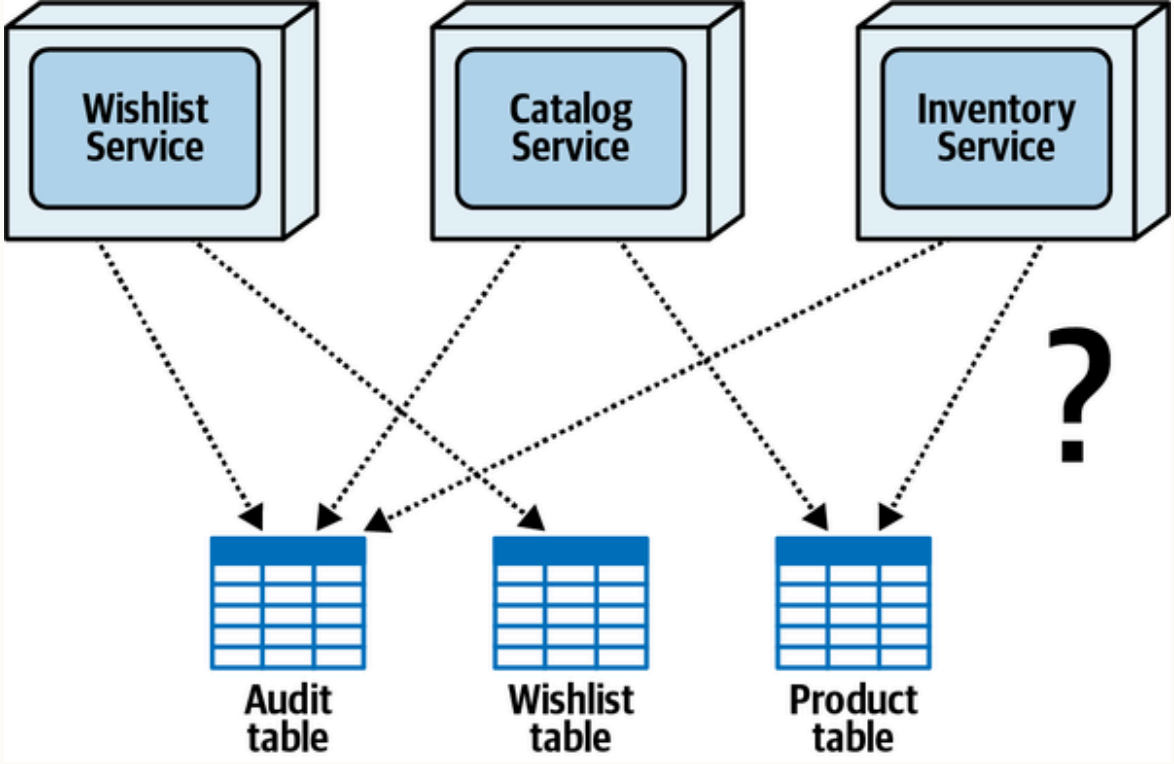


Figure 9-1. Once data is broken apart, tables must be assigned to services that own them

To further complicate matters, notice that the Wishlist Service writes to both the Audit table and the Wishlist table, the Catalog Service writes to the Audit table and the Product table, and the Inventory Service writes to the Audit table and the Product table. Suddenly, this simple real-world example makes assigning data ownership a complex and confusing task.

In this chapter, we unravel this complexity by discussing the three scenarios encountered when assigning data ownership to services (single ownership, common ownership, and joint ownership), and exploring techniques for resolving these scenarios, using Figure 9-1 as a common reference point.

Single Ownership Scenario

Single table ownership occurs when only one service writes to a table. This is the most straightforward of the data ownership scenarios and is relatively easy to resolve. Referring back to Figure 9-1, notice that the Wishlist table has only a single service that writes to it—the Wishlist Service.

In this scenario, it is clear that the Wishlist Service should be the owner of the Wishlist table (regardless of other services that need read-only access to the Wishlist table), see Figure 9-2. Notice that on the right side of this diagram, the Wishlist table becomes part of the bounded context of the Wishlist Service. This diagramming technique is an effective way to indicate table ownership and the bounded context formed between the service and its corresponding data.

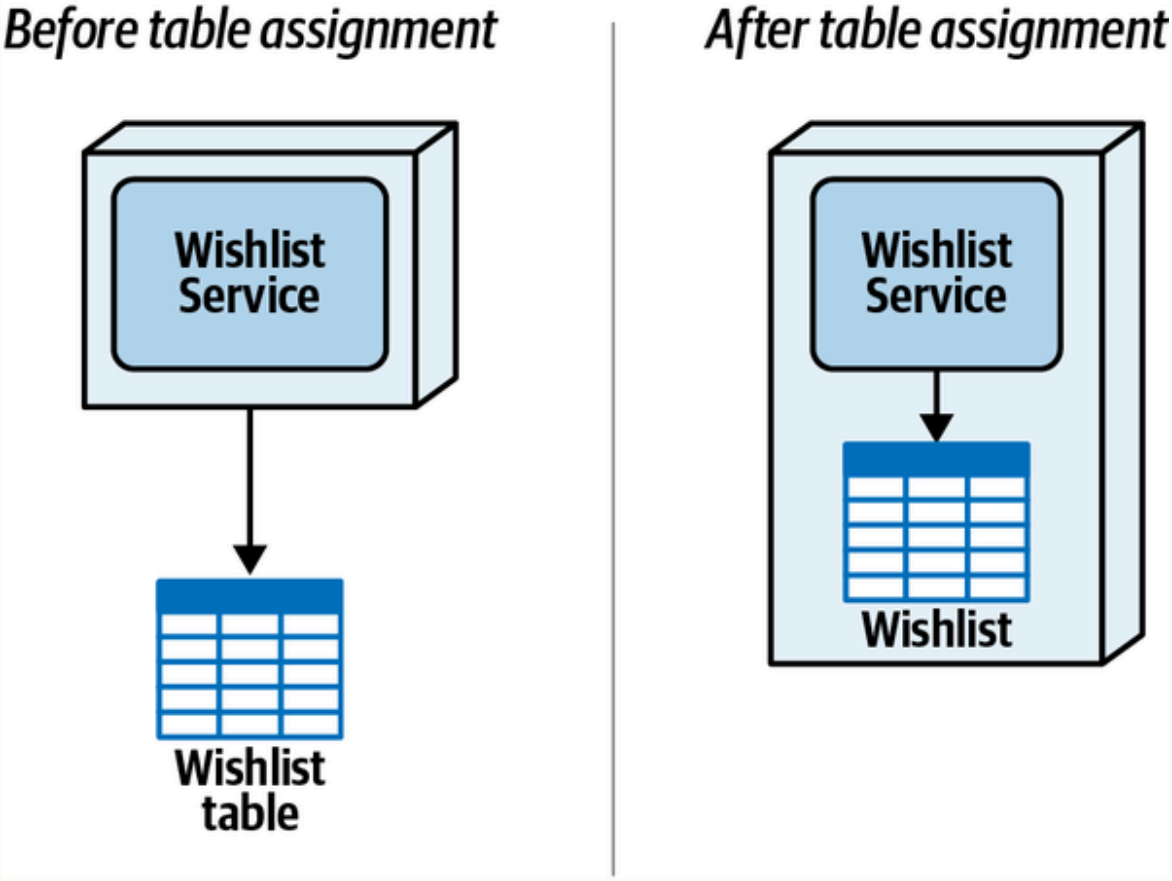


Figure 9-2. With single ownership, the service that writes to the table becomes the table owner

Because of the simplicity of this scenario, we recommend addressing single table ownership relationships first to clear the playing field in order to better address the more complicated scenarios that arise: common ownership and joint ownership.

Common Ownership Scenario

Common table ownership occurs when most (or all) of the services need to write to the same table. For example, Figure 9-1 shows that all services (Wishlist, Catalog, and Inventory) need to write to the Audit table to record the action performed by the user. Since all services need to write to the table, it's difficult to determine who should actually own the Audit table. While this simple example includes only three services, imagine a more realistic example where potentially hundreds (or even thousands) of services must write to the same Audit table.

The solution of simply putting the Audit table in a shared database or shared schema that is used by all services unfortunately reintroduces all of the data-sharing issues described at the beginning of Chapter 6, including change control, connection starvation, scalability, and fault tolerance. Therefore, another solution is needed to solve common data ownership.

A popular technique for addressing common table ownership is to assign a dedicated single service as the primary (and only) owner of that data, meaning only one service is responsi-

ble for writing data to the table. Other services needing to perform write actions would send information to the dedicated service, which would then perform the actual write operation on the table.

If no information or acknowledgment is needed by services sending the data, services can use persisted queues for asynchronous fire-and-forget messaging. Alternatively, if information needs to be returned to the caller based on a write action (such as returning a confirmation number or database key), services can use something like REST, gRPC, or request-reply messaging (pseudosynchronous) for a synchronous call.

Coming back to the Audit table example, notice in Figure 9-3 that the architect created a new Audit Service and assigned it ownership of the Audit table, meaning it is the only service that performs read or write actions on the table. In this example, since no return information is needed, the architect used asynchronous fire-and-forget messaging with a persistent queue so that the Wishlist Service, Catalog Service, and Inventory Service don't need to wait for the audit record to be written to the table. Making the queue persistent (meaning the message is stored on disk by the broker) provides guaranteed delivery in the event of a service or broker failure and helps ensure that no messages are lost.

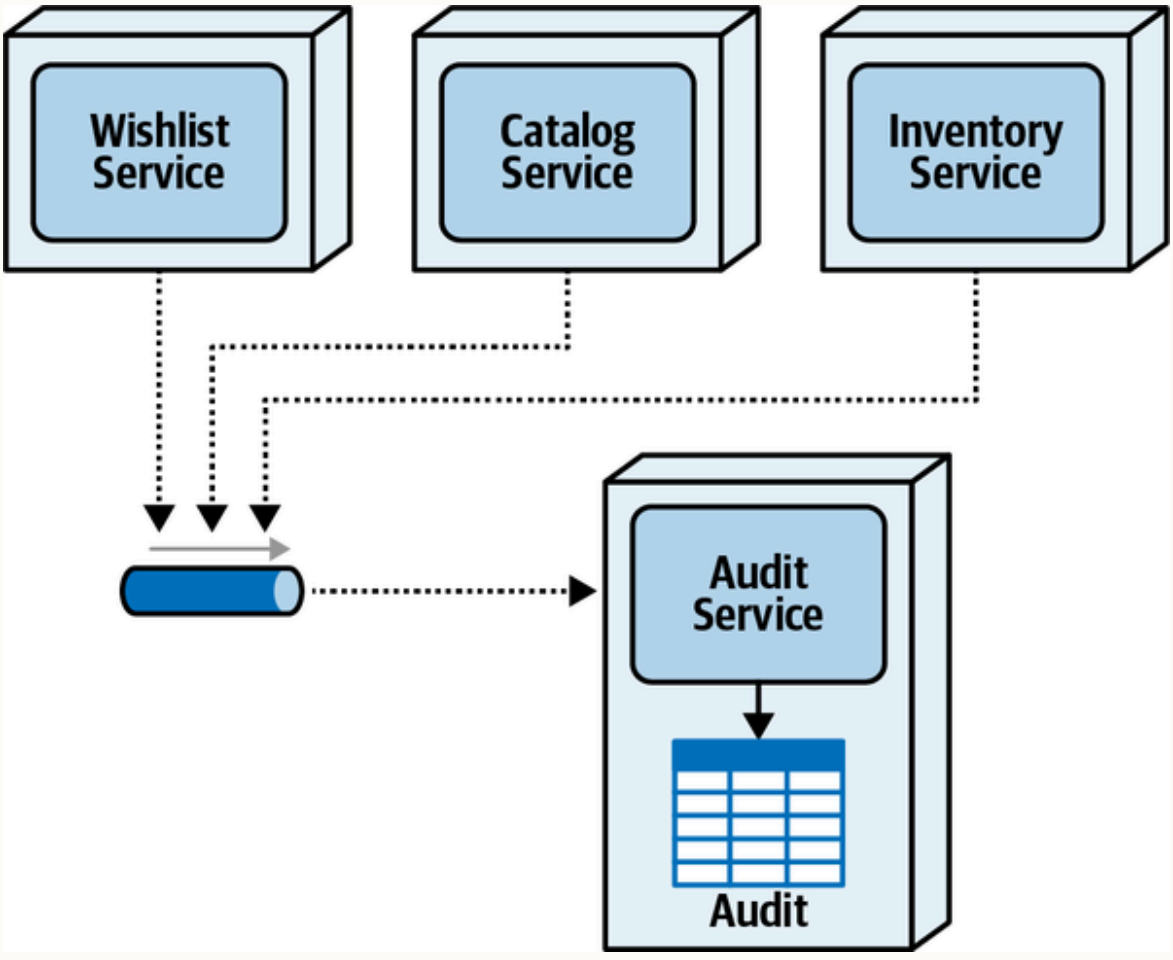


Figure 9-3. Common ownership uses a dedicated service owner

In some cases, it may be necessary for services to read common data they don't own. These read-only access techniques are described in detail in Chapter 10.

Joint Ownership Scenario

One of the more common (and complex) scenarios involving data ownership is *joint ownership*, which occurs when multiple services perform write actions on the same table. This scenario differs from the prior common ownership scenario in that with joint ownership, only a couple of services within the same domain write to the same table, whereas with common ownership, most or all of the services perform write operations on the same table. For example, notice in Figure 9-1 that all services perform write operations on the Audit table (common ownership), whereas only the Catalog and Inventory services perform write operations on the Product table (joint ownership).

Figure 9-4 shows the isolated joint ownership example from Figure 9-1. The Catalog Service inserts new products into the table, removes products no longer offered, and updates static product information as it changes, whereas the Inventory Service is responsible for reading and updating the current inventory for each product as products are queried, sold, or returned.

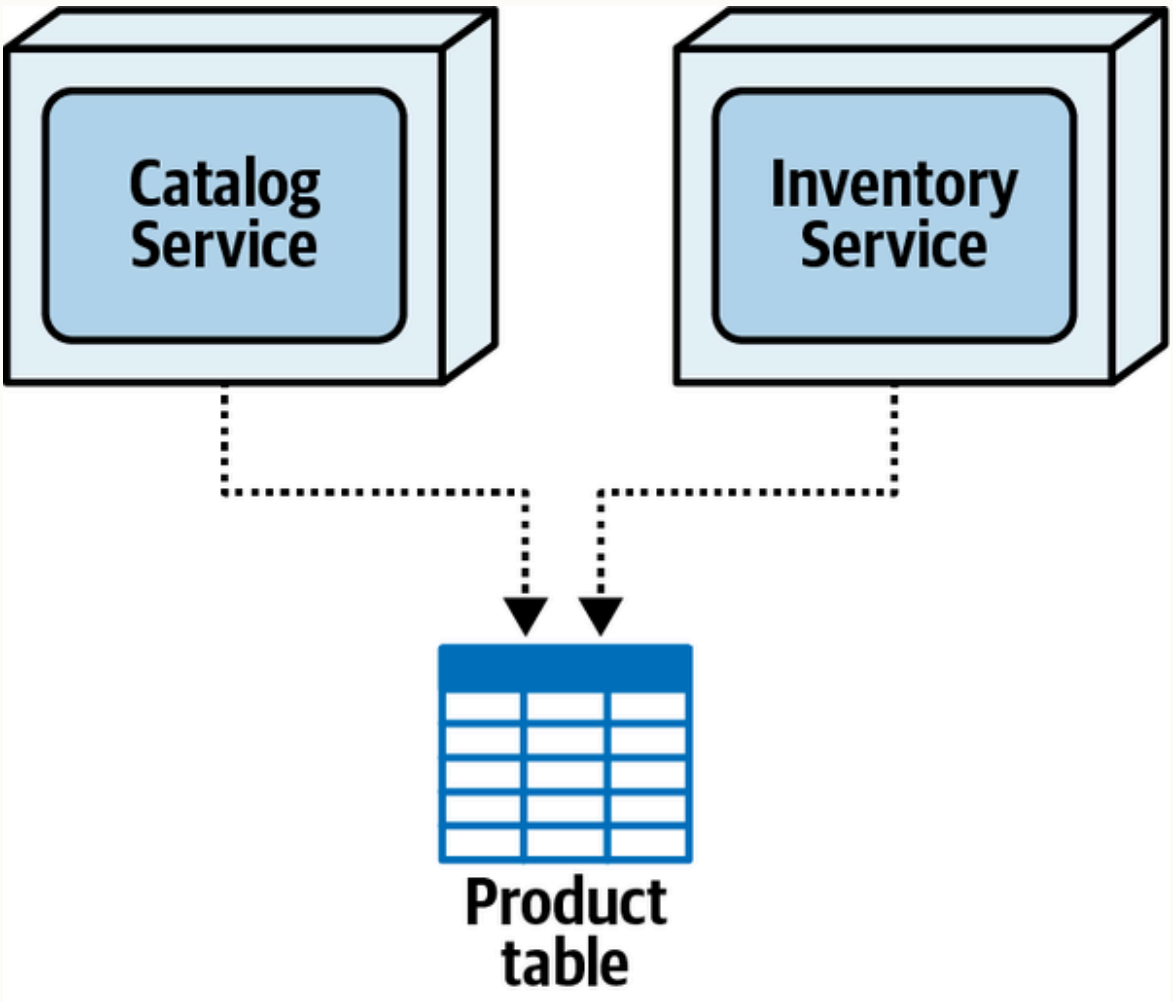


Figure 9-4. Joint ownership occurs when multiple services within the same domain perform write operations on the same table

Fortunately, several techniques exist to address this type of ownership scenario—the table split technique, the data domain technique, the delegation technique, and the service con-

solidation technique. Each is discussed in detail in the following sections.

Table Split Technique

The *table split technique* breaks a single table into multiple tables so that each service owns a part of the data it's responsible for. This technique is described in detail in the book *Refactoring Databases* and in the companion website.

To illustrate the table split technique, consider the Product table example illustrated in Figure 9-4. In this case, the architect or developer would first create a separate Inventory table containing the product ID (key) and the inventory count (number of items available), pre-populate the Inventory table with data from the existing Product table, then finally remove the inventory count column from the Product table. The source listing in Example 9-1 shows how this technique might be implemented using data definition language (DDL) in a typical relational database.

Example 9-1. DDL source code for splitting up the Product table and moving inventory counts to a new Inventory table

```
CREATE TABLE Inventory
(
product_id VARCHAR(10),
inv_cnt INT
);

INSERT INTO Inventory VALUES (product_id, inv_cnt)
AS SELECT product_id, inv_cnt FROM Product;

COMMIT;

ALTER TABLE Product DROP COLUMN inv_cnt;
```

Splitting the database table moves the joint ownership to a single table ownership scenario: the Catalog Service owns the data in the Product table, and the Inventory Service owns the data in the Inventory table. However, as shown in Figure 9-5, this technique requires communication between the Catalog Service and Inventory Service when products are created or removed to ensure the data remains consistent between the two tables.

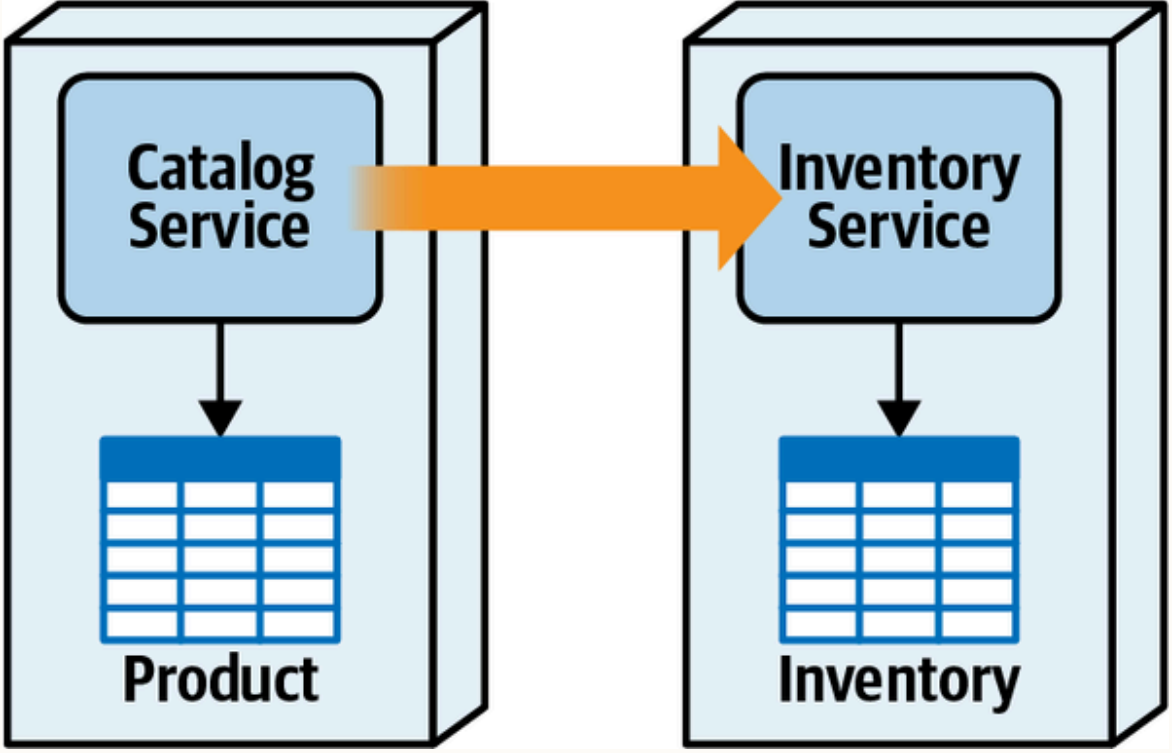


Figure 9-5. Joint ownership can be addressed by breaking apart the shared table

For example, if a new product is added, the Catalog Service generates a product ID and inserts the new product into the Product table. The Catalog Service then must send that new product ID (and potentially the initial inventory counts) to the Inventory Service. If a product is removed, the Catalog Service first removes the product from the Product table, then must notify the Inventory Service to remove the inventory row from the Inventory table.

Synchronizing data between split tables is not a trivial matter. Should communication between the Catalog Service and the Inventory Service be synchronous or asynchronous? What should the Catalog Service do when adding or removing a product and finding that the Inventory Service is not available? These are hard questions to answer, and are usually driven by the traditional *availability* verses *consistency* trade-off commonly found in distributed architectures. Choosing availability means that it's more important that the Catalog Service always be able to add or remove products, even though a corresponding inventory record may not be created in the Inventory table. Choosing consistency means that it's more important that the two tables always remain in sync with each other, which would cause a product creation or removal operation to fail if the Inventory Service is not available. Because network partitioning is necessary in distributed architectures, the CAP theorem states that only one of these choices (consistency or availability) is possible.

The type of communication protocol (synchronous versus asynchronous) also matters when splitting a table. Does the Catalog Service require a confirmation that the corresponding Inventory record is added when creating a new product? If so, then synchronous communication is required, providing better data consistency at the sacrifice of performance. If no confirmation is required, the Catalog Service can use asynchronous fire-and-forget commu-

nication, providing better performance at the sacrifice of data consistency. So many trade-offs to consider!

Table 9-1 summarizes the trade-offs associated with the table split technique for joint ownership.

TRADE-OFFS

Table 9-1. Joint ownership table split technique trade-offs

Advantages	Disadvantages
Preserves bounded context	Tables must be altered and restructured
Single data ownership	Possible data consistency issues
	No ACID transaction between table updates
	Data synchronization is difficult
	Data replication between tables may occur

Data Domain Technique

Another technique for joint ownership is to create a shared *data domain*. This is formed when data ownership is shared between the services, thus creating multiple owners for the table. With this technique, the tables shared by the same services are put into the same schema or database, therefore forming a broader bounded context between the services and the data.

Notice that Figure 9-6 looks close to the original diagram in Figure 9-4 with one noticeable difference—the data domain diagram has the Product table in a separate box outside the context of each owning service. This diagramming technique makes it clear that the table is not owned by or part of the bounded context of either service, but rather shared between them in a broader bounded context.

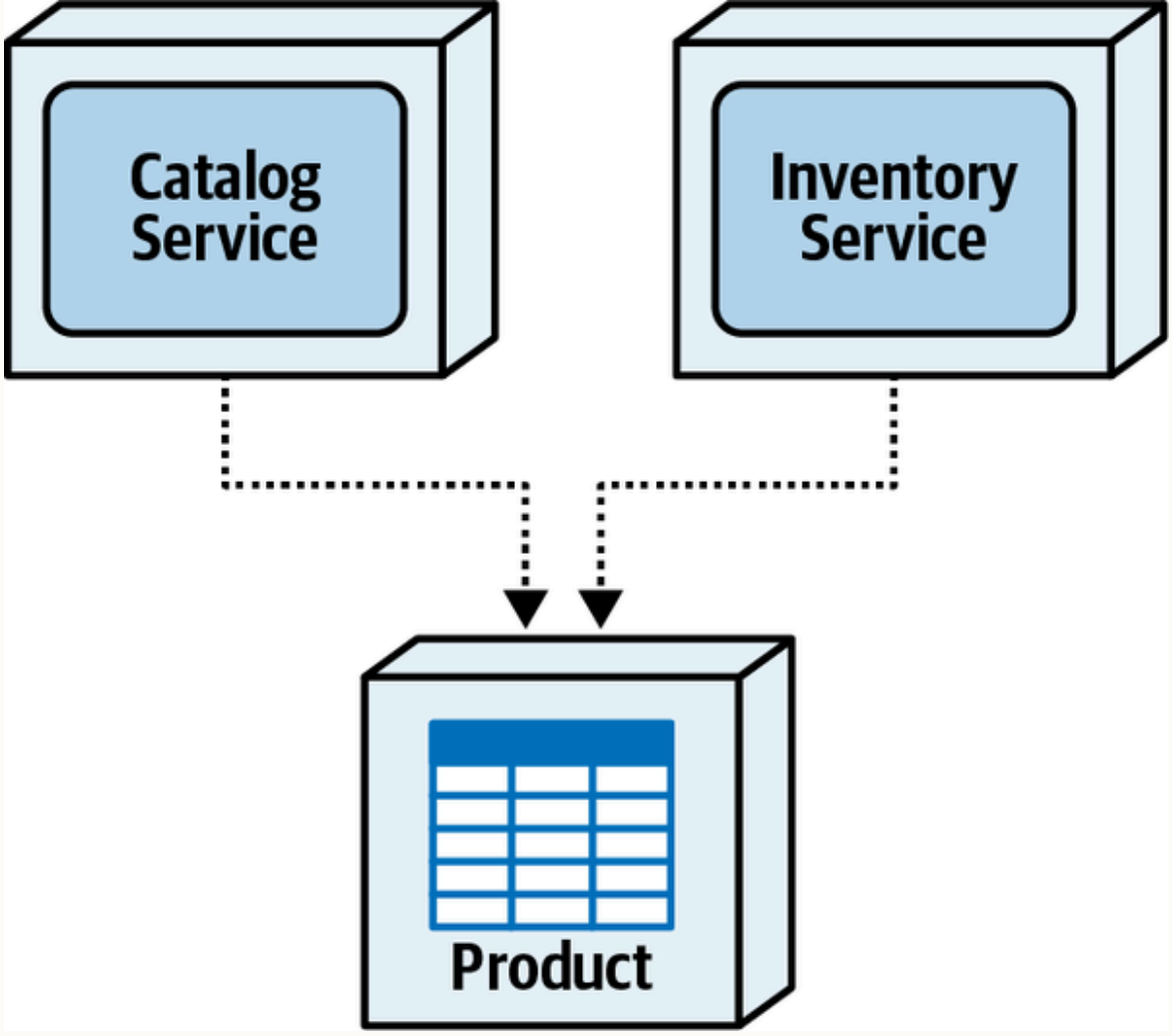


Figure 9-6. With joint ownership, services can share data by using the data domain technique (shared schema)

While data sharing is generally discouraged in distributed architectures (particularly with microservices), it does resolve some of the performance, availability, and data consistency issues found in other joint ownership techniques. Because the services are not dependent on each other, the Catalog Service can create or remove products without needing to coordinate with the Inventory Service, and the Inventory Service can adjust inventory without needing the Catalog Service. Both services become completely independent from each other.

TIP

When choosing the data domain technique, always reevaluate why separate services are needed since the data is common to each of the services. Justifications might include scalability differences, fault-tolerance needs, throughput differences, or isolating code volatility (see Chapter 7).

Unfortunately, sharing data in a distributed architecture introduces a number of issues, the first of these being increased effort for changes made to the structure of the data (such as changing the schema of a table). Because a broader bounded context is formed between the services and the data, changes to the shared table structures may require those changes

to be coordinated among multiple services. This increases development effort, testing scope, and deployment risk.

Another issue with the data domain technique with regard to data ownership is controlling which services have write responsibility to what data. In some cases, this might not matter, but if it's important to control write operations to certain data, additional effort is required to apply specific governance rules to maintain specific table or column write ownership.

Table 9-2 summarizes the trade-offs associated with the data domain technique for the joint ownership scenario.

TRADE-OFFS

Table 9-2. Joint ownership data-domain technique trade-offs

Advantages	Disadvantages
Good data access performance	Data schema changes involve more services
No scalability and throughput issues	Increased testing scope for data schema changes
Data remains consistent	Data ownership governance (write responsibility)
No service dependency	Increased deployment risk for data schema changes

---

Delegate Technique

An alternative method for addressing the joint ownership scenario is the *delegate technique*. With this technique, one service is assigned single ownership of the table and becomes the delegate, and the other service (or services) communicates with the delegate to perform updates on its behalf.

One of the challenges of the delegate technique is knowing which service to assign as the delegate (the sole owner of the table). The first option, called *primary domain priority*, assigns table ownership to the service that most closely represents the primary domain of the data—in other words, the service that does most of the primary entity CRUD operations for the particular entity within that domain. The second option, called *operational characteris-*

tics priority, assigns table ownership to the service needing higher operational architecture characteristics, such as performance, scalability, availability, and throughput.

To illustrate these two options and the corresponding trade-offs associated with each, consider the Catalog Service and Inventory Service joint ownership scenario shown in Figure 9-4. In this example, the Catalog Service is responsible for creating, updating, and removing products, as well as retrieving product information; the Inventory Service is responsible for retrieving and updating product inventory count as well as for knowing when to restock if inventory gets too low.

With the primary domain priority option, the service that performs most of the CRUD operations on the main entity becomes the owner of the table. As illustrated in Figure 9-7, since the Catalog Service performs most of the CRUD operations on product information, the Catalog Service would be assigned as the single owner of the table. This means that the Inventory service must communicate with the Catalog Service to retrieve or update inventory counts since it doesn't own the table.

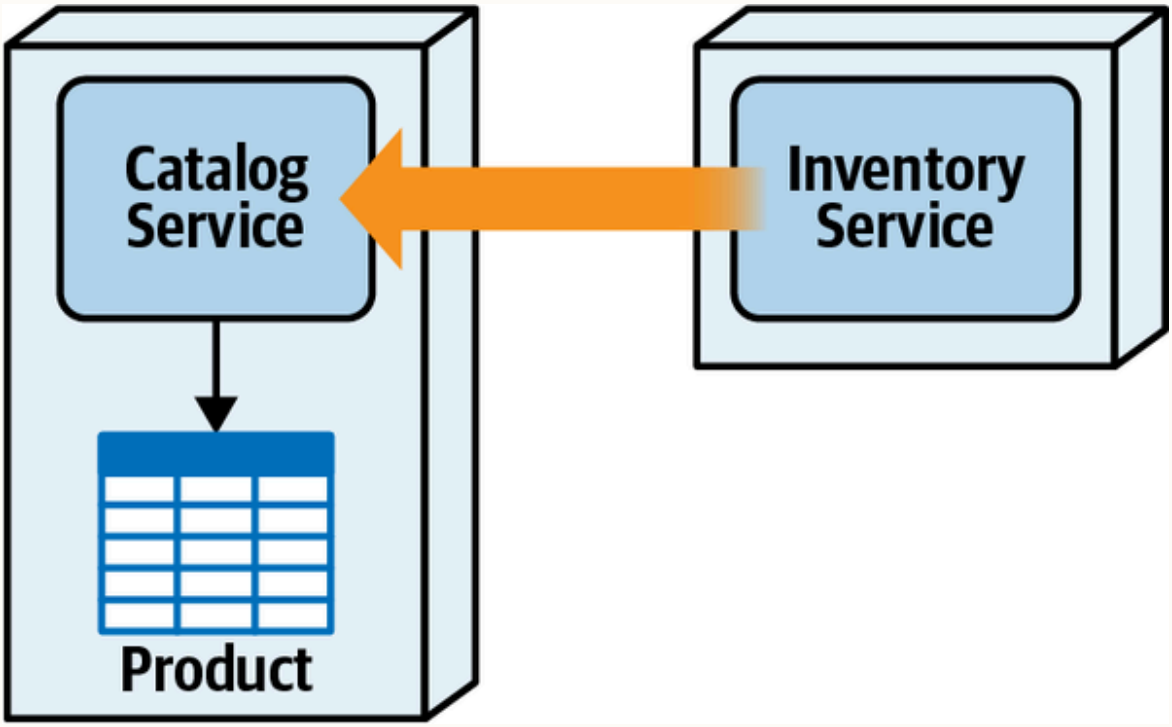


Figure 9-7. Table ownership is assigned to the Catalog service because of domain priority

Like the common ownership scenario described earlier, the delegate technique always forces interservice communication between the other services needing to update the data. Notice in Figure 9-7 that the Inventory Service must send inventory updates through some sort of remote access protocol to the Catalog Service so that it can perform the inventory updates and reads on behalf of the Inventory Service. This communication can either be synchronous or asynchronous. As always in software architecture, more trade-off analysis to consider.

With synchronous communication, the Inventory Service must wait for the inventory to be updated by the Catalog Service, which impacts overall performance but ensures data consistency. Using asynchronous communication to send inventory updates makes the Inventory Service perform much faster, but the data is only eventually consistent. Furthermore, with asynchronous communication, because an error can occur in the Catalog Service while trying to update inventory, the Inventory Service has no guarantee that the inventory was ever updated, impacting data integrity as well.

With the operational characteristics priority option, the ownership roles would be reversed because inventory updates occur at a much faster rate than static product data. In this case, table ownership would be assigned to the Inventory Service, the justification being that updating product inventory is a part of the frequent real-time transactional processing of purchasing products as opposed to the more infrequent administrative task of updating product information or adding and removing products (see Figure 9-8).

With this option, frequent updates to inventory counts can use direct database calls rather than remote access protocols, therefore making inventory operations much faster and more reliable. In addition, the most volatile data (inventory count) is kept highly consistent.

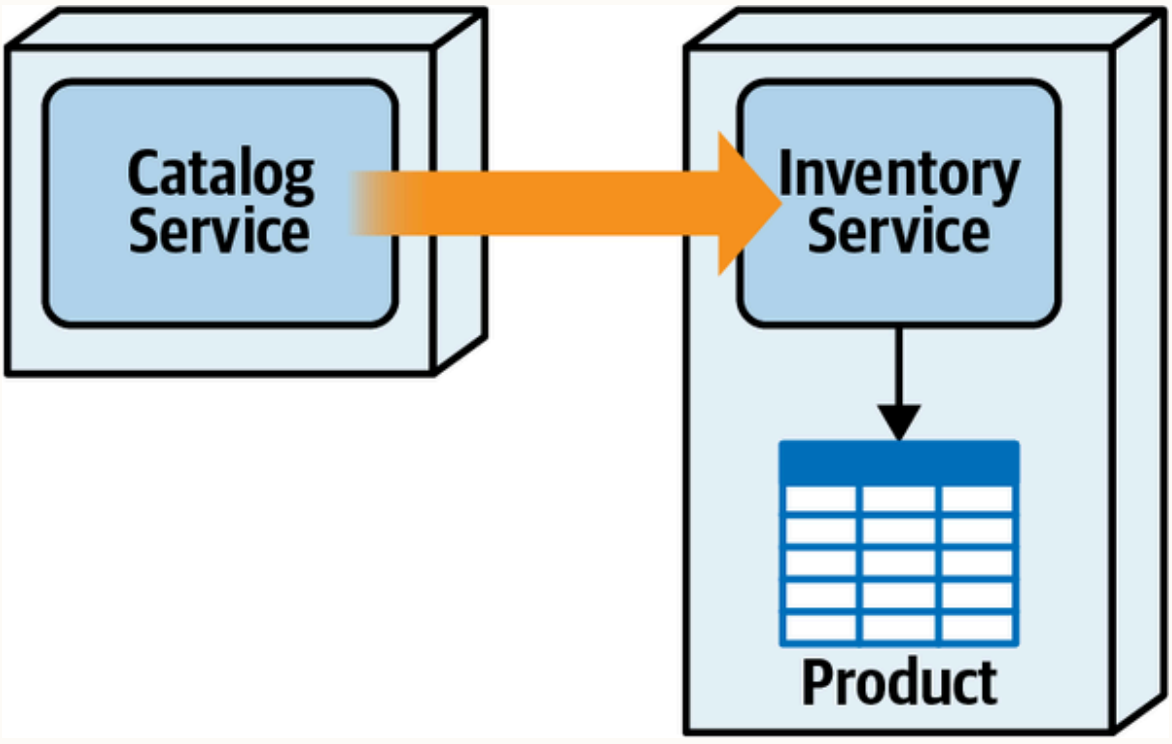


Figure 9-8. Table ownership is assigned to the Inventory Service because of operational characteristics priority

However, one major problem with the diagram illustrated in Figure 9-8 is that of domain management responsibility. The Inventory Service is responsible for managing product inventory, not the database activity (and corresponding error handling) for adding, removing, and updating static product information. For this reason, we usually recommend the domain priority option, and leveraging things like a replicated in-memory cache or a distributed cache to help address performance and fault-tolerance issues.

Regardless of which service is assigned as the delegate (sole table owner), the delegate technique has some disadvantages, the biggest being service coupling and the need for interservice communication. This in turn leads to other issues for nondelegate services, including the lack of an atomic transaction when performing write operations, low performance due to network and processing latency, and low fault tolerance. Because of these issues, the delegate technique is generally better suited for database write scenarios that do not require atomic transactions and that can tolerate eventual consistency through asynchronous communications.

Table 9-3 summarizes the overall trade-offs of the delegate technique.

TRADE-OFFS

Table 9-3. Joint ownership delegate technique trade-offs

Advantages	Disadvantages
Forms single table ownership	High level of service coupling
Good data schema change control	Low performance for nonowner writes
Abstracts data structures from other services	No atomic transaction for nonowner writes
	Low fault tolerance for nonowner services

Service Consolidation Technique

The delegate approach discussed in the prior section highlights the primary issue associated with joint ownership—service dependency. The *service consolidation technique* resolves service dependency and addresses joint ownership by combining multiple table owners (services) into a single consolidated service, thus moving joint ownership into a single ownership scenario (see Figure 9-9).

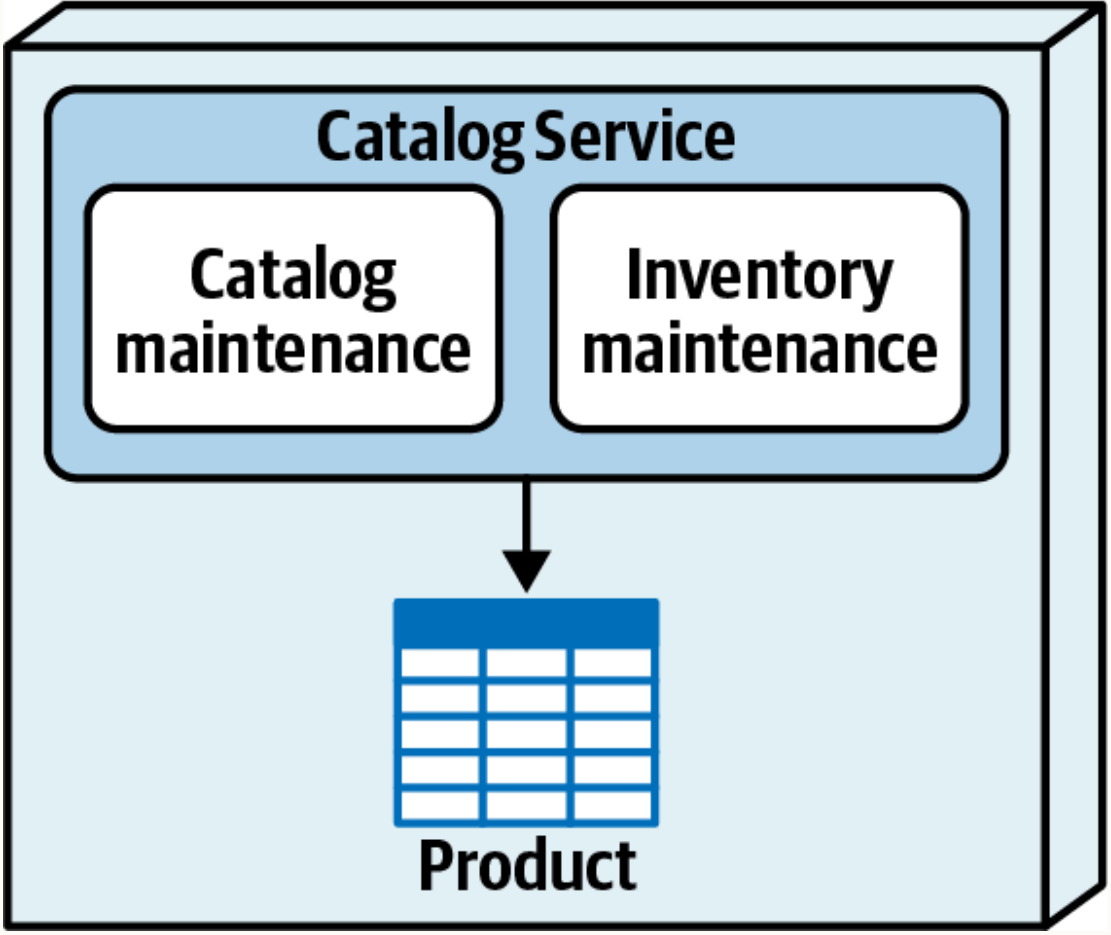


Figure 9-9. Table ownership is resolved by combining services

Like the data domain technique, this technique resolves issues associated with service dependencies and performance, while at the same time addressing the joint ownership problem. However, like the other techniques, it has its share of trade-offs as well.

Combining services creates a more coarse-grained service, thereby increasing the overall testing scope as well as overall deployment risk (the chance of breaking something else in the service when a new feature is added or a bug is fixed). Consolidating services might also impact overall fault tolerance since all parts of the service fail together.

Overall scalability is also impacted when using the service consolidation technique because *all* parts of the service must scale equally, even though some functionality might not need to scale at the same level as other functionality. For example, in Figure 9-9, the catalog maintenance functionality (what used to be in a separate `Catalog` service) must unnecessarily scale to meet the high demands of the inventory retrieval and update functionality.

Table 9-4 summarizes the overall trade-offs of the service consolidation technique.

TRADE-OFFS

Table 9-4. Joint ownership service consolidation technique trade-offs

Advantages	Disadvantages
Preserves atomic transactions	More coarse-grained scalability
Good overall performance	Less fault tolerance
	Increased deployment risk
	Increased testing scope

Data Ownership Summary

Figure 9-10 shows the resulting table ownership assignments from Figure 9-1 after applying the techniques described in this section. For the single table scenario involving the Wishlist Service, we simply assigned ownership to the Wishlist Service, forming a tight bounded context between the service and the table. For the common ownership scenario involving the audit table, we created a new Audit Service, with all other services sending an asynchronous message to a persisted queue. Finally, for the more complex joint ownership scenario involving the product table with the Catalog Service and Inventory Service, we chose to use the delegate technique, assigning single ownership of the product table to the Catalog Service, with the Inventory Service sending update requests to the Catalog Service.

Once table ownership has been assigned to services, an architect must then validate the table ownership assignments by analyzing business workflows and their corresponding transaction requirements.

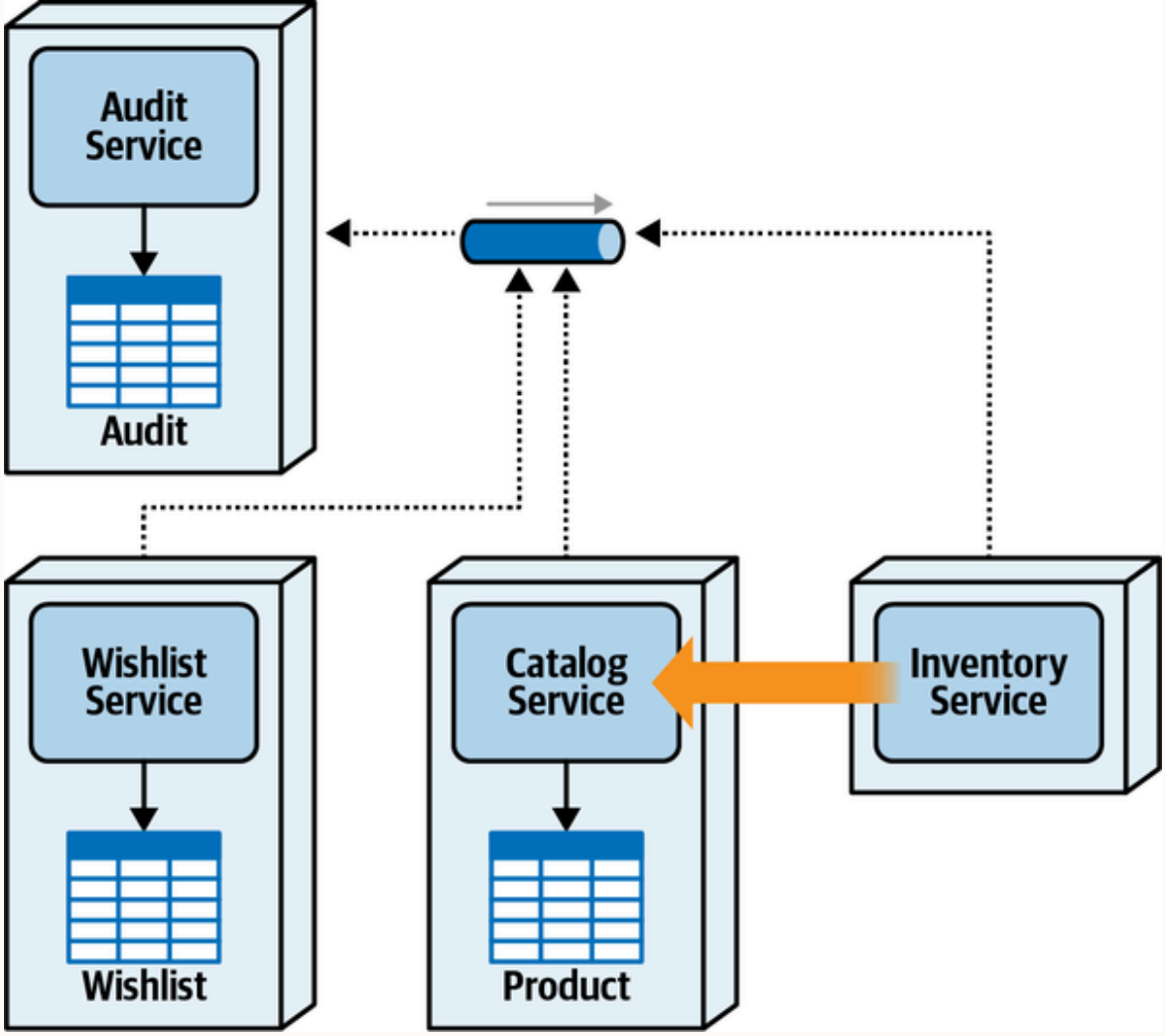


Figure 9-10. Resulting data ownership using delegate technique for joint ownership

Distributed Transactions

When architects and developers think about transactions, they usually think about a single atomic unit of work where multiple database updates are either committed together or all rolled back when an error occurs. This type of atomic transaction is commonly referred to as an *ACID transaction*. As noted in Chapter 6, ACID is an acronym describing the basic properties of an atomic single-unit-of-work database transaction: atomicity, consistency, isolation, and durability.

To understand how distributed transactions work and the trade-offs involved with using a distributed transaction, it's necessary to fully understand the four properties of an ACID transaction. We firmly believe that without an understanding of ACID transactions, an architect cannot perform the necessary trade-off analysis for knowing when (and when not to) use a distributed transaction. Therefore, we will dive into the details of an ACID transaction first, then describe how they differ from distributed transactions.

Atomicity means a transaction must either commit or roll back *all* of its updates in a single unit of work, regardless of the number of updates made during that transaction. In other words, all updates are treated as a collective whole, so all changes either get committed or

get rolled back as one unit. For example, assume registering a customer involves inserting customer profile information into a Customer Profile table, inserting credit card information into a Wallet table, and inserting security-related information into a Security table. Suppose the profile and credit card information are successfully inserted, but the security information insert fails. With atomicity, the profile and credit card inserts would be rolled back, keeping the database tables in sync.

Consistency means that during the course of a transaction, the database would never be left in an inconsistent state or violate any of the integrity constraints specified in the database. For example, during an ACID transaction, the system cannot add a detail record (such as an item) without first adding the corresponding summary record (such as an order). Although some databases defer this check until commit time, in general programmers cannot violate consistency constraints such as a foreign-key constraint during the course of a transaction.

Isolation refers to the degree to which individual transactions interact with each other. Isolation protects uncommitted transaction data from being visible to other transactions during the course of the business request. For example, during the course of an ACID transaction, when the customer profile information is inserted into the Customer Profile table, no other services outside of the ACID transaction scope can access the newly inserted information until the entire transaction is committed.

Durability means that once a successful response from a transaction commit occurs, it is guaranteed that *all* data updates are permanent, regardless of further system failures.

To illustrate an ACID transaction, suppose a customer registering for the Sysops Squad application enters all of their profile information, the electronic products they want covered under the support plan, and their billing information on a single user interface screen. This information is then sent to the single Customer Service, as shown in Figure 9-11, which then performs all of the database activity associated with the customer registration business request.

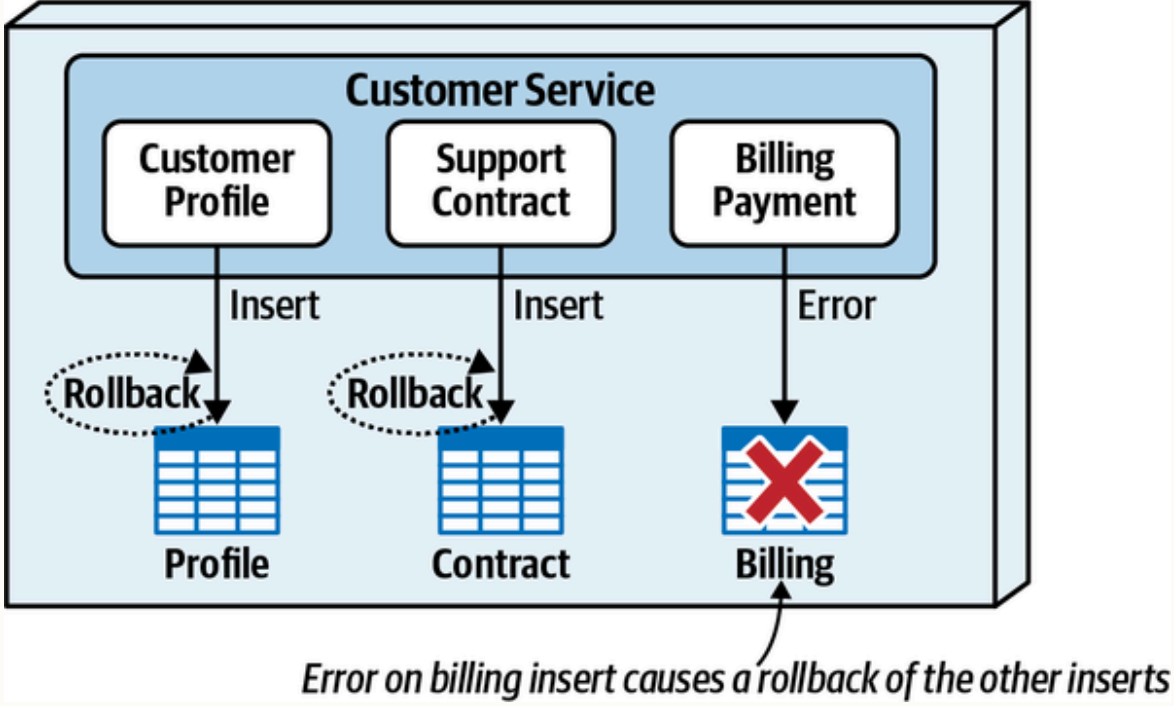


Figure 9-11. With ACID transactions, an error on the billing insert causes a rollback to the other table inserts

First, notice that with an ACID transaction, because an error occurred when trying to insert the billing information, both the profile information and support contract information that were previously inserted are now rolled back (that's the *atomicity* and *consistency* parts of ACID). While not illustrated in the diagram, data inserted into each table during the course of the transaction is not visible to other requests (that's the *isolation* part of ACID).

Note that ACID transactions can exist *within the context of each service* in a distributed architecture, but only if the corresponding database supports ACID properties as well. Each service can perform its own commits and rollbacks to the tables it owns within the scope of the atomic business transaction. However, if the business request spans multiple services, the entire business request itself cannot be an ACID transaction—rather, it becomes a *distributed transaction*.

Distributed transactions occur when an atomic business request containing multiple database updates is performed by separately deployed remote services. Notice in Figure 9-12 that the same request for a new customer registration (denoted by the laptop image representing the customer making the request) is now spread across three separately deployed services—a Customer Profile Service, a Support Contract Service, and a Billing Payment Service.

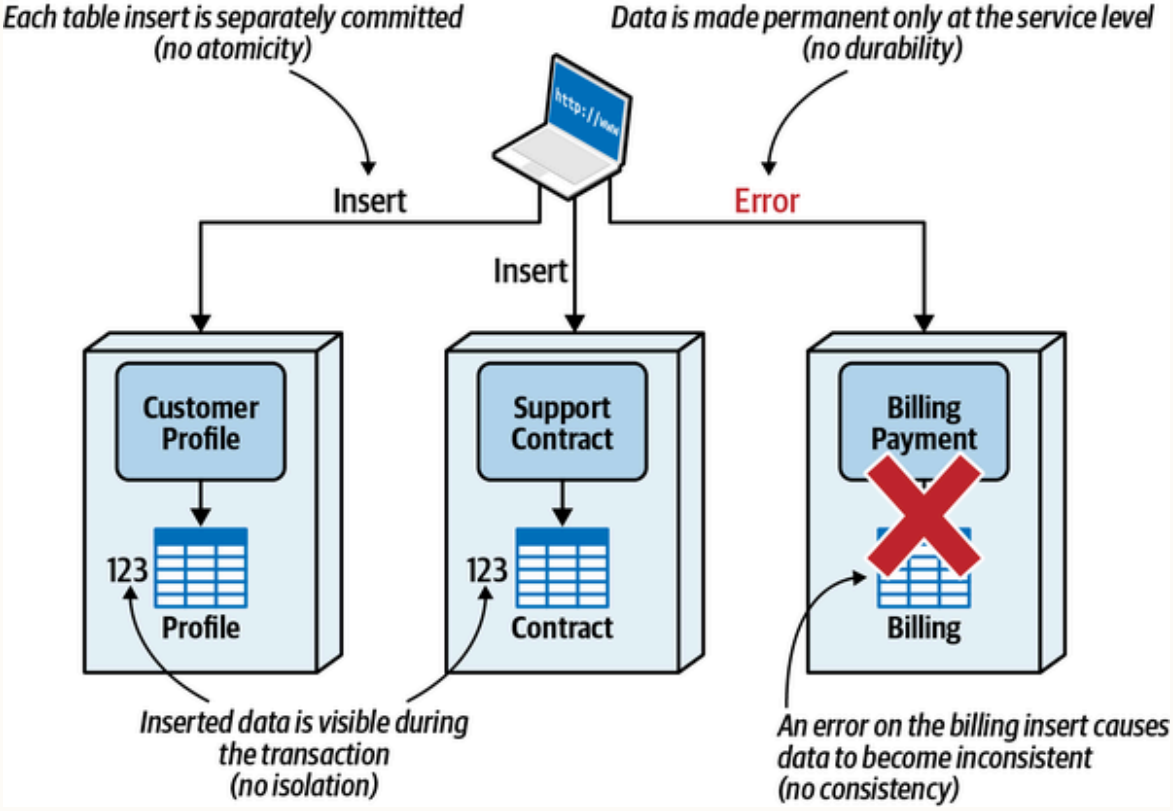


Figure 9-12. Distributed transactions do not support ACID properties

As you can see, distributed transactions do not support ACID properties.

Atomicity is not supported because each separately deployed service commits its own data and performs only one part of the overall atomic business request. In a distributed transaction, atomicity is bound to the *service*, not the *business request* (such as customer registration).

Consistency is not supported because a failure in one service causes the data to be out of sync between the tables responsible for the business request. As shown in Figure 9-12, since the Billing Payment Service insert failed, the Profile table and Contract table are now out of sync with the Billing table (we'll show how to address these issues later in this section). Consistency is also impacted because traditional relational database constraints (such as a foreign key always matching a primary key) cannot be applied during each individual service commit.

Isolation is not supported because once the Customer Profile Service inserts the profile data in the course of a distributed transaction to register a customer, that profile information is available to any other service or request, even though the customer registration process (the current transaction) hasn't completed.

Durability is not supported across the business request—it is supported for only each individual service. In other words, any individual commit of data does not ensure that *all* data within the scope of the entire business transaction is permanent.

Instead of ACID, distributed transactions support something called *BASE*. In chemistry, an *acid* substance and a *base* substance are exactly the opposite. The same is true with atomic and distributed transactions—ACID transactions are opposite of BASE transactions. BASE describes the properties of a distributed transaction: basic availability, soft state, and eventual consistency.

Basic availability (the "BA" part of BASE) means that all of the services or systems in the distributed transaction are expected to be available to participate in the distributed transaction. While asynchronous communication can help decouple services and address availability issues associated with the distributed transaction participants, it unfortunately impacts how long it will take the data to become consistent for the atomic business transaction (see eventual consistency later in this section).

Soft state (the *S* part of BASE) describes the situation where a distributed transaction is in progress and the state of the atomic business request is not yet complete (or in some cases not even known). In the customer registration example shown in Figure 9-12, soft state occurs when the customer profile information is inserted (and committed) in the Profile table, but the support contract and billing information are not. The unknown part of soft state can occur if, using the same example, all three services work in parallel to insert their corresponding data—the exact state of the atomic business request is not known at any point in time until *all three* services report back that the data has been successfully processed. In the case of a workflow using asynchronous communication (see Chapter 11), the in-progress or final state of the distributed transaction is usually difficult to determine.

Eventual consistency (the *E* part of BASE) means that given enough time, all parts of the distributed transaction will complete successfully and all of the data is in sync with one another. The type of eventual consistency pattern used and the way errors are handled dictates how long it will take for all of the data sources involved in the distributed transaction to become consistent.

The next section describes the three types of eventual consistency patterns and the corresponding trade-offs associated with each pattern.

Eventual Consistency Patterns

Distributed architectures rely heavily on eventual consistency as a trade-off for better operational architecture characteristics such as performance, scalability, elasticity, fault tolerance, and availability. While there are numerous ways to achieve eventual consistency between data sources and systems, the three main patterns in use today are the background synchronization pattern, orchestrated request-based pattern, and the event-based pattern.

To better describe each pattern and illustrate how they work, consider again the customer registration process from the Sysops Squad application we discussed earlier in Figure 9-13. In this example, three separate services are involved in the customer registration process: a Customer Profile Service that maintains basic profile information, a Support Contract Service that maintains products covered under the Sysops Squad repair plan for each customer, and a Billing Payment Service that charges the customer for the support plan. Notice in the figure that customer 123 is a subscriber to the Sysops Squad service, and therefore has data in each of the corresponding tables owned by each service.

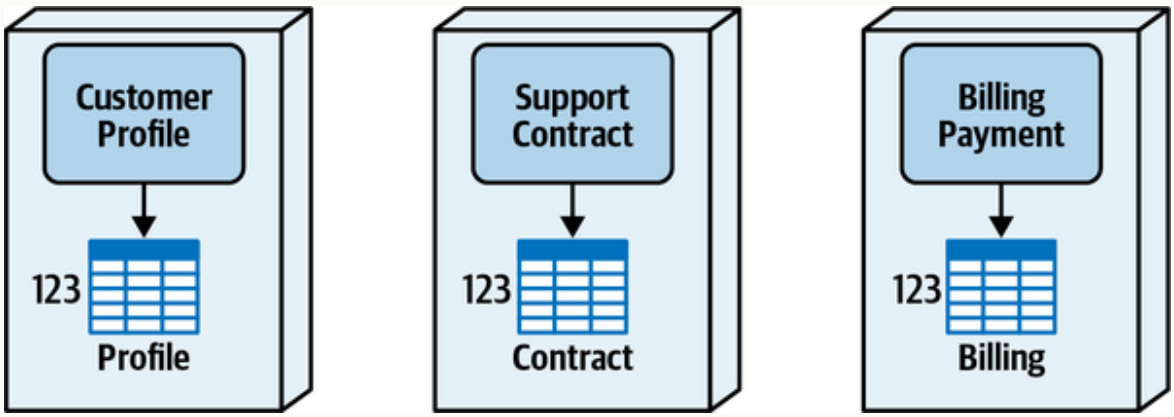


Figure 9-13. Customer 123 is a subscriber in the Sysops Squad application

Customer 123 decides they are no longer interested in the Sysops Squad support plan, so they unsubscribe from the service. As shown in Figure 9-14, the Customer Profile Service receives this request from the user interface, removes the customer from the Profile table, and returns a confirmation to the customer that they are successfully unsubscribed and will no longer be billed. However, data for that customer still exists in the Contract table owned by the Support Contract Service and the Billing table owned by the Billing Payment Service.

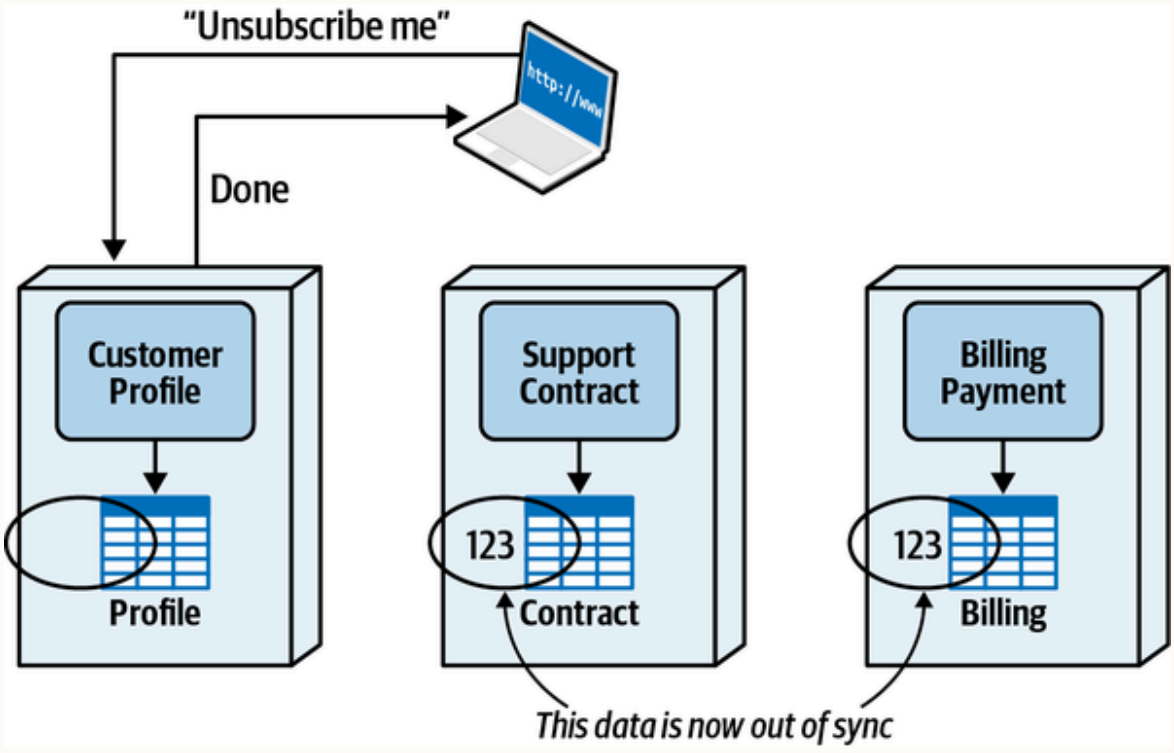


Figure 9-14. Data is out of sync after the customer unsubscribes from the support plan

We will use this scenario to describe each of the eventual consistency patterns for getting all of the data in sync for this atomic business request.

Background Synchronization Pattern

The *background synchronization pattern* uses a separate external service or process to periodically check data sources and keep them in sync with one another. The length of time for data sources to become eventually consistent using this pattern can vary based on whether the background process is implemented as a batch job running sometime in the middle of the night, or a service that wakes up periodically (say, every hour) to check the consistency of the data sources.

Regardless of how the background process is implemented (nightly batch or periodic), this pattern usually has the longest length of time for data sources to become consistent. However, in many cases data sources do not need to be kept in sync immediately. Consider the customer unsubscribe example in Figure 9-14. Once a customer unsubscribes, it really doesn't matter that the support contract and billing information for that customer still exists. In this case, eventual consistency done during the night is a sufficient amount of time to get the data in sync.

One of the challenges of this pattern is that the background process used to keep all the data in sync must know what data has changed. This can be done through an event stream, a database trigger, or reading data from source tables and aligning target tables with the source data. Regardless of the technique used to identify changes, the background process must have knowledge of all the tables and data sources involved in the transaction.

Figure 9-15 illustrates the use of the background synchronization pattern for the Sysops Squad unregister example. Notice that at 11:23:00 the customer issues a request to unsubscribe from the support plan. The Customer Profile Service receives the request, removes the data, and one second later (11:23:01) responds back to the customer that they have been successfully unsubscribed from the system. Then, at 23:00 the background batch synchronization process starts. The background synchronization process detects that customer 123 has been removed either through event streaming or primary table versus secondary table deltas, and deletes the data from the Contract and Billing tables.

This pattern is good for overall responsiveness because the end user doesn't have to wait for the entire business transaction to complete (in this case, unsubscribing from the support plan). But, unfortunately, some serious trade-offs with this eventual consistency pattern.

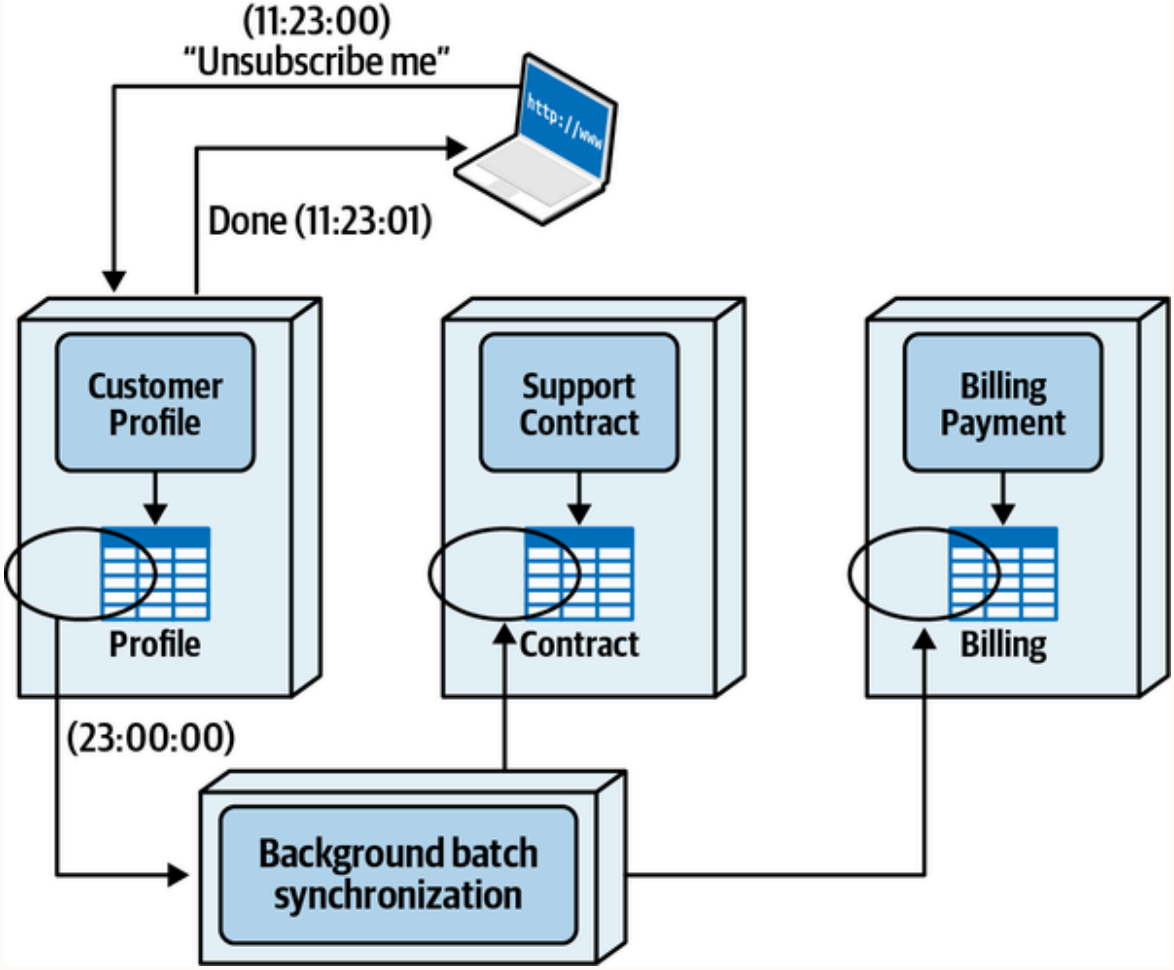


Figure 9-15. The background synchronization pattern uses an external process to ensure data consistency

The biggest disadvantage of the background synchronization pattern is that it couples all of the data sources together, thus breaking every bounded context between the data and the services. Notice in Figure 9-16 that the background batch synchronization process must have write access to each of the tables owned by the corresponding services, meaning that all of the tables effectively have shared ownership between the services and the background synchronization process.

This shared data ownership between the services and the background synchronization process is riddled with issues, and emphasizes the need for tight bounded contexts within a distributed architecture. Structural changes made to the tables owned by each service (changing a column name, dropping a column, and so on) must also be coordinated with an external background process, making changes difficult and time-consuming.

In addition to difficulties with change control, problems occur with regard to duplicated business logic as. In looking at Figure 9-15, it might seem fairly straightforward that the background process would simply perform a `DELETE` operation on all rows in the Contract and Billing tables containing customer 123. However, certain business rules may exist within these services for the particular operation.

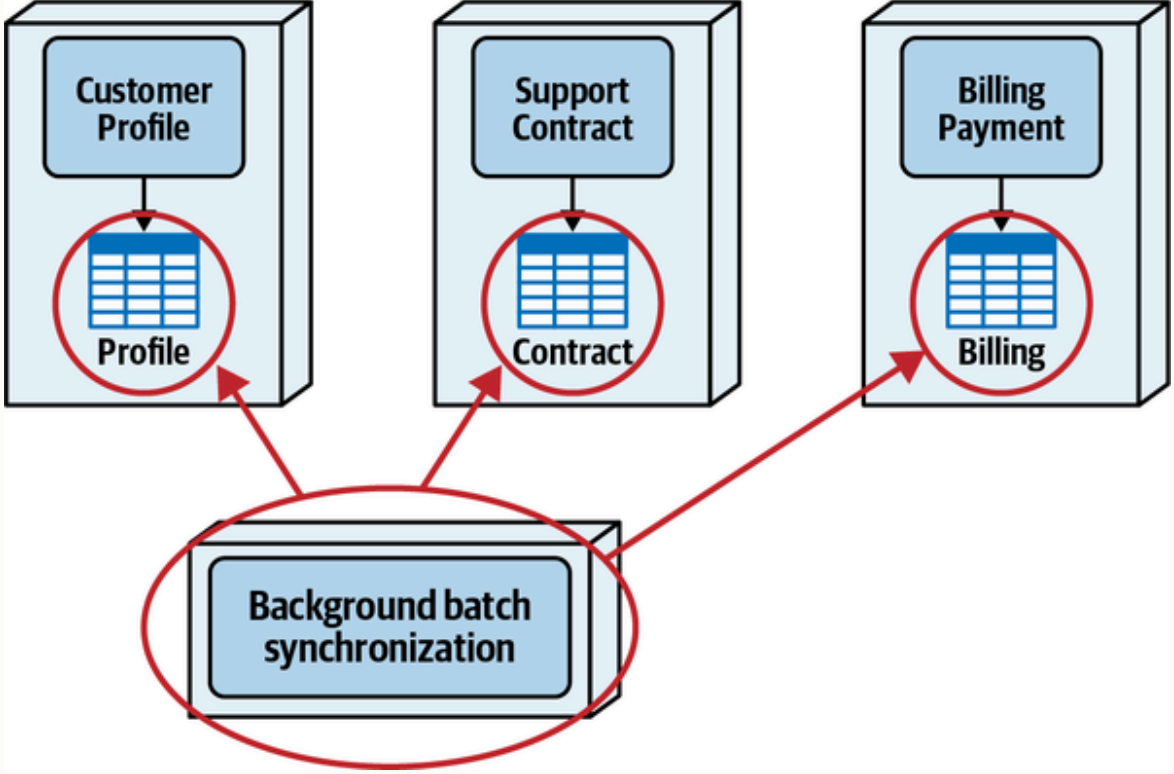


Figure 9-16. The background synchronization pattern is coupled to the data sources, therefore breaking the bounded context and data ownership

For example, when a customer unsubscribes, their existing support contracts and billing history are kept for three months in the event the customer decides to resubscribe to the support plan. Therefore, rather than deleting the rows in those tables, a `remove_date` column is set with a long value representing the date the rows should be removed (a zero value in this column indicates an active customer). Both services check the `remove_date` daily to determine which rows should be removed from their respective tables. The question is, where is that business logic located? The answer, of course, is in the Support Contract and Billing Payment Services—oh, and *also the background batch process!*

The background synchronization eventual consistency pattern is not suitable for distributed architectures requiring tight bounded contexts (such as microservices) where the coupling between data ownership and functionality is a critical part of the architecture. Situations where this pattern is useful are closed (self-contained) heterogeneous systems that don't communicate with each other or share data.

For example, consider a contractor order entry system that accepts orders for building materials, and another separate system (implemented in a different platform) that does contractor invoicing. Once a contractor orders supplies, a background synchronization process moves those orders to the invoicing system to generate invoices. When a contractor changes an order or cancels it, the background synchronization process moves those changes to the invoicing system to update the invoices. This is a good example of systems becoming *eventually consistent*, with the contractor order always in sync between the two systems.

Table 9-5 summarizes the trade-offs for the background synchronization pattern for eventual consistency.

TRADE-OFFS

Table 9-5. Background synchronization pattern trade-offs

Advantages	Disadvantages
Services are decoupled	Data source coupling
Good responsiveness	Complex implementation
	Breaks bounded contexts
	Business logic may be duplicated
	Slow eventual consistency

Orchestrated Request-Based Pattern

A common approach for managing distributed transactions is to make sure all of the data sources are synchronized during the course of the business request (in other words, while the end user is waiting). This approach is implemented through what is known as the *orchestrated request-based pattern*.

Unlike the previous background synchronization pattern or the event-based pattern described in the next section, the orchestrated request-based pattern attempts to process the entire distributed transaction *during the business request*, and therefore requires some sort of orchestrator to manage the distributed transaction. The orchestrator, which can be a designated existing service or a new separate service, is responsible for managing all of the work needed to process the request, including knowledge of the business process, knowledge of the participants involved, multicasting logic, error handling, and contract ownership.

One way to implement this pattern is to designate one of the primary services (assuming there is one) to manage the distributed transaction. This technique, illustrated in Figure 9-17, designates one of the services to take on the role as orchestrator in addition to its other responsibilities, which in this case is the Customer Profile Service.

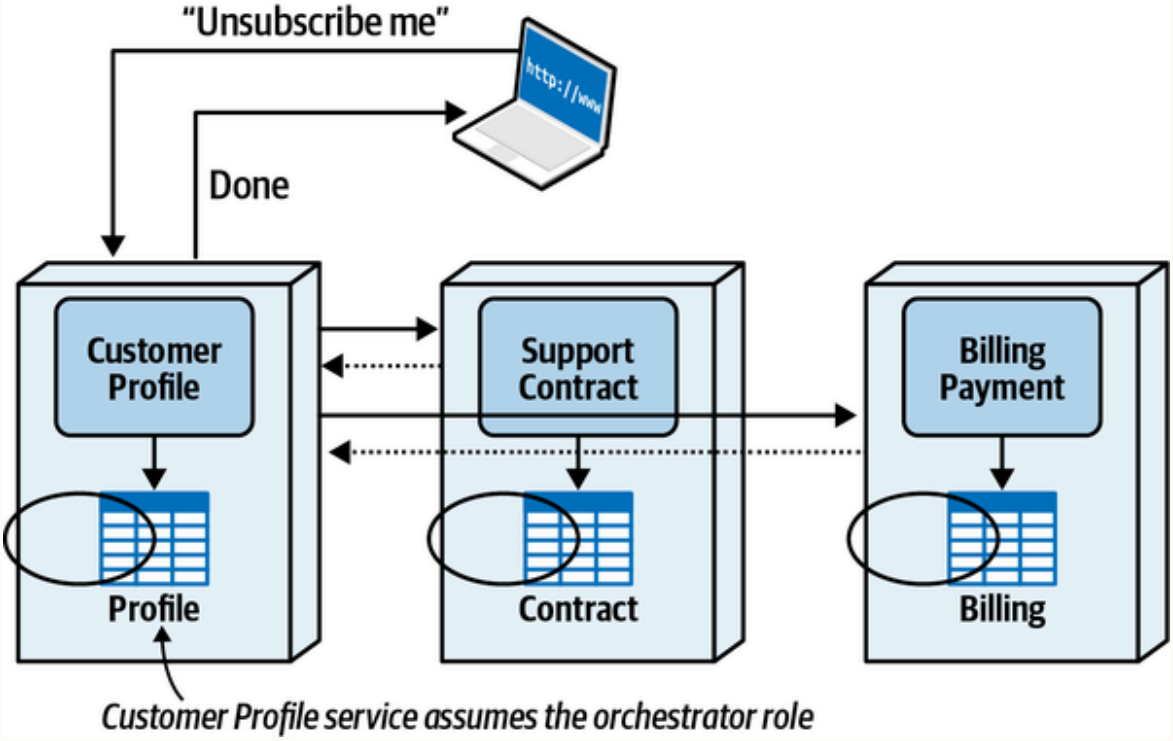


Figure 9-17. The Customer Profile Service takes on the role of an orchestrator for the distributed transaction

Although this approach avoids the need for a separate orchestration service, it tends to overload the responsibilities of the service designated as the distributed transaction orchestrator. In addition to the role of an orchestrator, the designated service managing the distributed transaction must perform its own responsibilities as well. Another drawback to this approach is that it lends itself to tight coupling and synchronous dependencies between services.

The approach we generally prefer when using the orchestrated request-based pattern is to use a dedicated orchestration service for the business request. This approach, illustrated in Figure 9-18, frees up the Customer Profile Service from the responsibility of managing the distributed transaction and places that responsibility on a separate orchestration service.

We will use this separate orchestration service approach to describe how this eventual consistency pattern works and the corresponding trade-offs with this pattern.

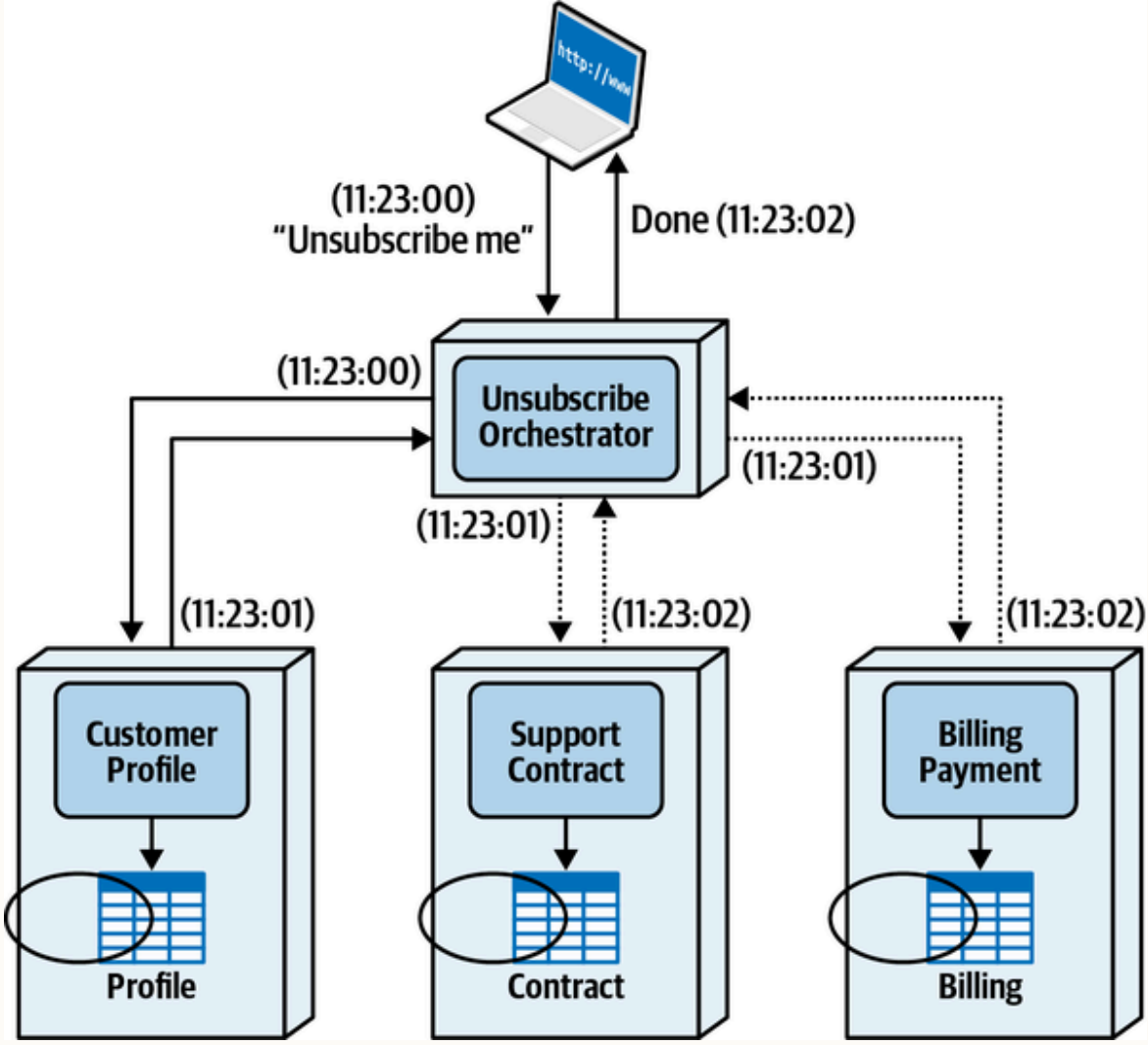


Figure 9-18. A dedicated orchestration service takes on the role of an orchestrator for the distributed transaction

Notice that at 11:23:00 the customer issues a request to unsubscribe from the Sysops Squad support plan. The request is received by the Unsubscribe Orchestrator Service, which then forwards the request synchronously to the Customer Profile Service to remove the customer from the Profile table. One second later, the Customer Profile Service sends back an acknowledgment to the Unsubscribe Orchestrator Service, which then sends parallel requests (either through threads or some sort of asynchronous protocol) to both the Support Contract and Billing Payment Services. Both of these services process the unsubscribe request, and then send an acknowledgment back one second later to the Unsubscribe Orchestrator Service indicating they are done processing the request. Now that all data is in sync, the Unsubscribe Orchestrator Service responds back to the client at 11:23:02 (two seconds after the initial request was made), letting the customer know they were successfully unsubscribed.

The first trade-off to observe is that the orchestration approach generally favors data consistency over responsiveness. Adding a dedicated orchestration service not only adds additional network hops and service calls, but depending on whether the orchestrator executes calls serially or in parallel, additional time is needed for the back-and-forth communication between the orchestrator and the services it's calling.

Response time could be improved in Figure 9-18 by executing the Customer Profile request at the same time as the other services, but we chose to do that operation synchronously for error handling and consistency reasons. For example, if the customer could not be deleted from the Profile table because of an outstanding billing charge, no other action is needed to reverse the operations in the Support Contract and Billing Payment Services. This represents another example of consistency over responsiveness.

Besides responsiveness, the other trade-off with this pattern is complex error handling. While the orchestrated request-based pattern might seem straightforward, consider what happens when the customer is removed from the Profile table and Contract table, but an error occurs when trying to remove the billing information from the Billing table, as illustrated in Figure 9-19. Since the Profile and Support Contract Services individually committed their operations, the Unsubscribe Orchestrator Service must now decide what action to take *while the customer is waiting for the request to be processed*:

1. Should the orchestrator send the request again to the Billing Payment Service for another try?
2. Should the orchestrator perform a compensating transaction and have the Support Contract and Customer Profile Services reverse their update operations?
3. Should the orchestrator respond to the customer that an error occurred and to wait a bit before trying again, while trying to repair the inconsistency?
4. Should the orchestrator ignore the error in hopes that some other process will deal with the issue and respond to the customer that they have been successfully unsubscribed?

This real-world scenario creates a messy situation for the orchestrator. Because this *is* the eventual consistency pattern used, there is no other means to correct the data and get things back in sync (therefore negating options 3 and 4 in the preceding list). In this case, the only real option for the orchestrator is to try to reverse the distributed transaction—in other words, issue a *compensating update* to reinsert the customer in the Profile table and set the `remove_date` column in the Contract table back to zero. This would require the orchestrator to have all of the necessary information to reinsert the customer, and that no side effects occur when creating a new customer (such as initializing the billing information or support contracts).

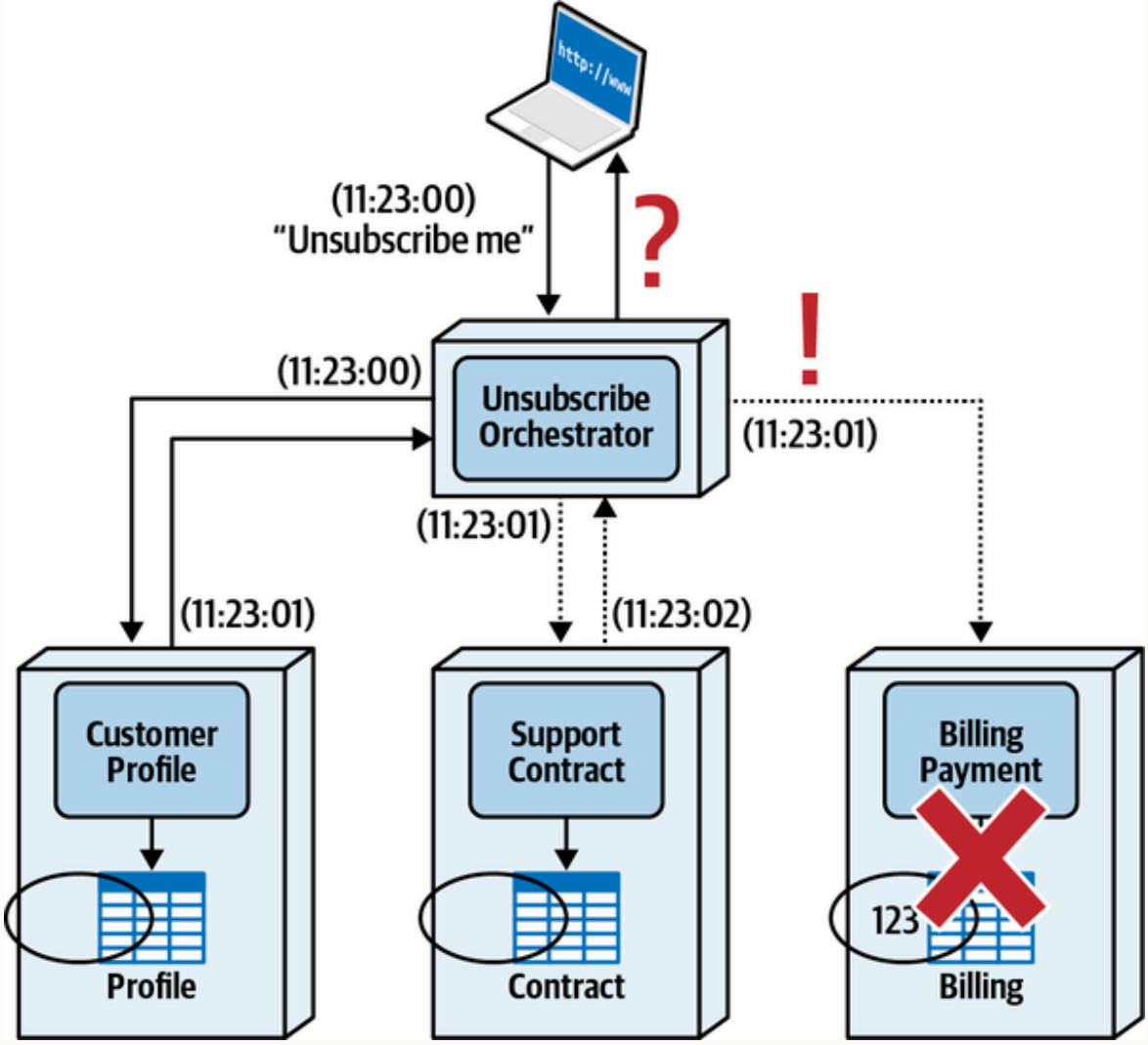


Figure 9-19. Error conditions are very hard to address when using the orchestrated request-based pattern

Another complication with compensating transactions in a distributed architecture is failures that occur during compensation. For example, suppose a compensating transaction was issued to the Customer Profile Service to reinsert the customer, and that operation failed. Now what? Now the data is really out of sync, and there's no other service or process around to repair the problem. Most cases like these typically require human intervention to repair the data sources and get them back in sync. We go into more details about compensating transactions and transactional sagas in "Transactional Saga Patterns".

Table 9-6 summarizes the trade-offs for the orchestrated request-based pattern for eventual consistency.

TRADE-OFFS

Table 9-6. Orchestrated request-based pattern trade-offs

Advantages	Disadvantages
Services are decoupled	Slower responsiveness
Timeliness of data consistency	Complex error handling
Atomic business request	Usually requires compensating transactions

Event-Based Pattern

The *event-based pattern* is one of the most popular and reliable eventual consistency patterns for most modern distributed architectures, including microservices and event-driven architectures. With this pattern, events are used in conjunction with an asynchronous publish-and-subscribe (pub/sub) messaging model to post events (such as `customer unsubscribed`) or command messages (such as `unsubscribe customer`) to a topic or event stream. Services involved in the distributed transaction listen for certain events and respond to those events.

The eventual consistency time is usually short for achieving data consistency because of the parallel and decoupled nature of the asynchronous message processing. Services are highly decoupled from one another with this pattern, and responsiveness is good because the service triggering the eventual consistency event doesn't have to wait for the data synchronization to occur before returning information to the customer.

Figure 9-20 illustrates how the event-based pattern for eventual consistency works. Notice that the customer issues the unsubscribe request to the Customer Profile Service at 11:23:00. The Customer Profile Service receives the request, removes the customer from the Profile table, publishes a message to a message topic or event stream, and returns information one second later letting the customer know they were successfully unsubscribed. At around the same time this happens, both the Support Contract and Billing Payment Services receive the unsubscribe event and perform whatever functionality is needed to unsubscribe the customer, making all the data sources eventually consistent.

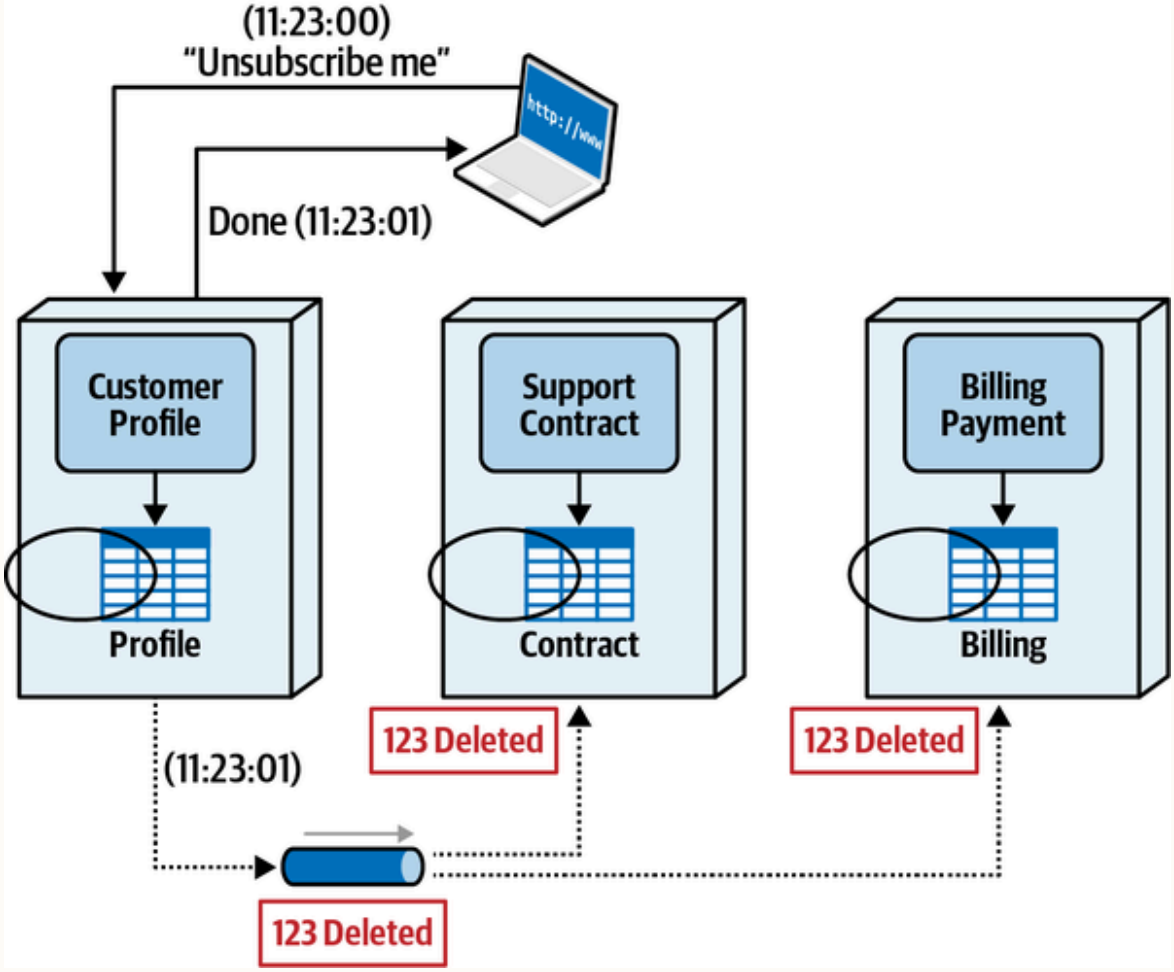


Figure 9-20. The event-based pattern uses asynchronous publish-and-subscribe messaging or event streams to achieve eventual consistency

For implementations using standard topic-based publish-and-subscribe messaging (such as ActiveMQ, RabbitMQ, AmazonMQ, and so on), services responding to the event must be set up as *durable subscribers* to ensure no messages are lost if the message broker or the service receiving the message fails. A durable subscriber is similar in concept to persistent queues in that the subscriber (in this case, the Support Contract Service and Billing Payment Service) does not need to be available at the time the message is published, and subscribers are guaranteed to receive the message once they become available. In the case of event streaming implementations, the message broker (such as Apache Kafka) must always persist the message and make sure it is available in the topic for a reasonable amount of time.

The advantages of the event-based pattern are responsiveness, timeliness of data consistency, and service decoupling. However, similar to all eventual consistency patterns, the main trade-off of this pattern is error handling. If one of the services (for example, the Billing Payment Service illustrated in Figure 9-20) is not available, the fact that it is a durable subscriber means that *eventually* it will receive and process the event when it does become available. However, if the service is processing the event and fails, things get complicated quickly.

Most message brokers will try a certain number of times to deliver a message, and after repeated failures by the receiver, the broker will send the message to a *dead letter queue* (*DLQ*). This is a configurable destination where the event is stored until an automated

process reads the message and tries to fix the problem. If it can't be repaired programmatically, the message is then typically sent to a human for manual processing.

Table 9-7 lists the trade-offs for the event-based pattern for eventual consistency.

TRADE-OFFS

Table 9-7. Event-based pattern trade-offs

Advantages	Disadvantages
Services are decoupled	Complex error handling
Timeliness of data consistency	
Fast responsiveness	

Sysops Squad Saga: Data Ownership for Ticket Processing

Tuesday, January 18, 09:14



After talking with Dana and learning about data ownership and distributed transaction management, Sydney and Addison quickly realized that breaking apart data and assigning data ownership to form tight bounded contexts wasn't possible without both teams collaborating on the solution.

"No wonder nothing ever seems to work around here," observed Sydney. "We've always had issues and arguments between us and the database team, and now I see the results of our company treating us as two separate teams."

"Exactly," said Addison. "I'm glad we are working more closely with the data team now. So, from what Dana said, the service that performs write actions on the data table owns the table, regardless of what other services need to access the data in a read-only manner. In that case, looks like the User Maintenance Service needs to own the data."

Sydney agreed, and Addison created a general architecture decision record describing what to do for single-table ownership scenarios:

ADR: Single Table Ownership for Bounded Contexts

Context
When forming bounded contexts between services and data, tables must be assigned ownership to a particular service or group of services.

Decision
When only one service writes to a table, that table will be assigned ownership to that service. Furthermore, services requiring read-only access to a table in another bounded context cannot directly access the database or schema containing that table.

Per the database team, table ownership is defined as the service that performs write operations on a table. Therefore, for single table ownership scenarios, regardless of how many other services need to access the table, only one service is ever assigned an owner, and that owner is the service that maintains the data.

Consequences
Depending on the technique used, services requiring read-only access to a table in another bounded context may incur performance and fault-tolerance issues when accessing data in a different bounded context.

Now that Sydney and Addison better understood table ownership and how to form bounded contexts between the service and the data, they started to work on the survey functionality. The Ticket Completion Service would write the timestamp the ticket was completed and the expert who performed the job to the survey table. The Survey Service would write the timestamp the survey was sent to the customer, and also insert all of the survey results once the survey is received.

"This isn't so hard now that I better understand bounded contexts and table ownership," said Sydney.

"OK, let's move on to the survey functionality," said Addison.

"Oops," said Sydney. "Both the Ticket Completion Service and the Survey Service write to the Survey table."

"That's what Dana called joint-table ownership," said Addison.

"So, what are our options?" asked Sydney.

"Since splitting up the table won't work, it really leaves us with only two options," said Addison. "We can use a common data domain so that both services own the data, or we can

use the delegate technique and assign only one service as the owner."

"I like the common data domain. Let both services write to the table and share a common schema," said Sydney.

"Except that won't work in this scenario," said Addison. "The Ticket Completion Service is already talking to the common ticketing data domain. Remember, a service can't connect to multiple schemas."

"Oh, right," said Sydney. "Wait, I know, just add the survey tables to the ticketing data domain schema."

"But now we are starting to combine all the tables back together." said Addison. "Pretty soon we'll be right back to a monolithic database again."

"So what do we do?" asked Sydney.

"Wait, I think I see a good solution here," said Addison. "You know how the Ticket Completion Service has to send a message to the Survey Service anyway to kick off the survey process once a ticket is complete? What if we passed in the necessary data along with that message so that the Survey Service can insert the data when it creates the customer survey?"

"That's brilliant," said Sydney. "That way, the Ticket Completion doesn't need any access to the Survey table."

Addison and Sydney agreed that the Survey Service would own the Survey table, and would use the delegation technique to pass data when the table notifies the Survey Service to kick off the survey process as illustrated in Figure 9-21. Addison wrote an architecture decision record for this decision.

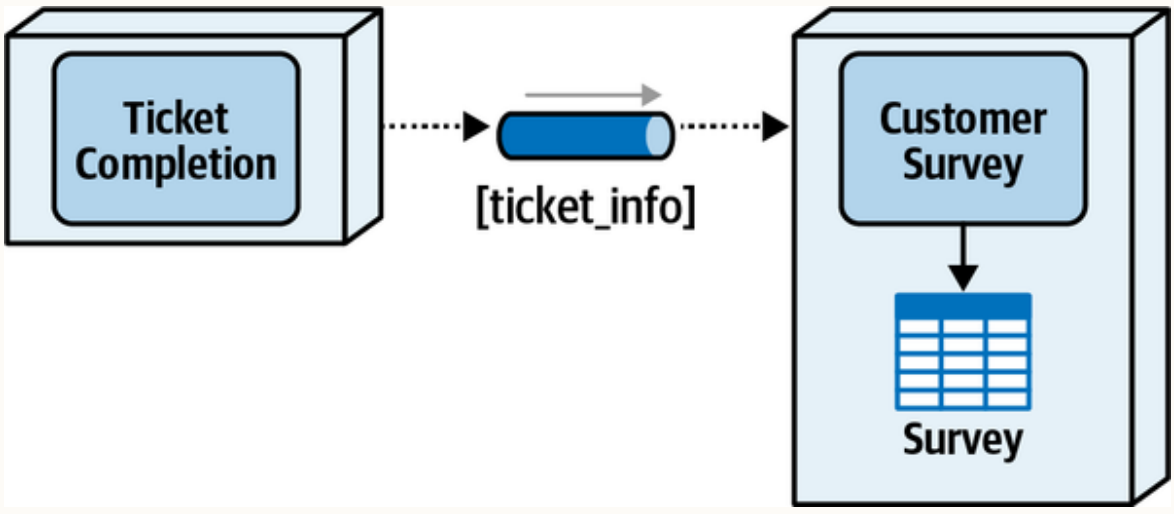


Figure 9-21. Survey Service owns the data using the delegation technique

ADR: Survey Service Owns the Survey Table

Context
Both the Ticket Completion Service and the Survey Service write to the Survey table. Because this is a joint ownership scenario, the alternatives are to use a common shared data domain or use the delegation technique. Table splitting is not an option because of the structure of the Survey table.

Decision
The Survey Service will be the single owner of the Survey table, meaning it is the only service that can perform write operations to that table.

Once a ticket is marked as complete and is accepted by the system, the Ticket Completion Service needs to send a message to the Survey Service to kick off the customer survey processing. Since the Ticket Completion Service is already sending a notification event, the necessary ticket information can be passed along with that event, thus eliminating the need for the Ticket Completion Service to have any access to the Survey table.

Consequences
All of the necessary data that the Ticket Completion Service needs to insert into the Survey table will need to be sent as part of the payload when triggering the customer survey process.

In the monolithic system, the ticket completion inserted the survey record as part of the completion process. With this decision, the creation of the survey record is a separate activity from the ticket creation process and is now handled by the Survey Service.